



DevOps: Report

Docker-compose and microservices hands-on

Table of contents

Report on the Docker-compose and microservices hands-on exercise	3
Platform used of the exercise	3
Host:.....	3
docker and docker compose version:	3
Diagrams of the Architecture:	4
Analysis of the content of the status records.....	5
Output:	5
Which disc space and uptime you measured?	5
Both services measured the same data since they are executed on the same host. Furthermore, the disk space is almost the same only rounding issues lead to a slight difference.	5
How relevant are those measurements?	5
What should be done better?	5
Analysis and comparison of persistent storage	6
Teacher's instruction for cleaning up the persistent storage	6
What was difficult?	6
What were the main problems?.....	7

GeorgSeng

Remakes on the technology used:..... 7

Service 1 / Storage 7

Service 2 7

Report on the Docker-compose and microservices hands-on exercise

The goal of the exercise is to reinforce knowledge of Docker as well as building images. Furthermore, the goal was to study two different approaches to how data can be stored: in a shared folder/file or on a dedicated service.

```
git clone -b exercise1 https://github.com/GeorgSeng/devops-course.git
```

Platform used of the exercise

The platform used for this exercise is an HP laptop running Linux. For Docker, the implementation provided by Podman was used. This choice was made simply because it is recommended on Fedora Linux and works out of the box. The benefit of Podman is that there is no need for a daemon running with root permissions. Podman provides an API that emulates the behavior of Docker, as seen below.

Host:

HW: HP ZBook

OS: Fedora Linux 42 (KDE Plasma Desktop Edition)

Kernel: 6.16.7-200.fc42.x86_64 (64-bit)

Processors: 20 x 13th Gen Intel Core i7-13800H

Memory: 32GiB of RAM

docker and docker compose version:

```
→ storage git:(exercise1) X docker --version
```

Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.

podman version 5.6.1

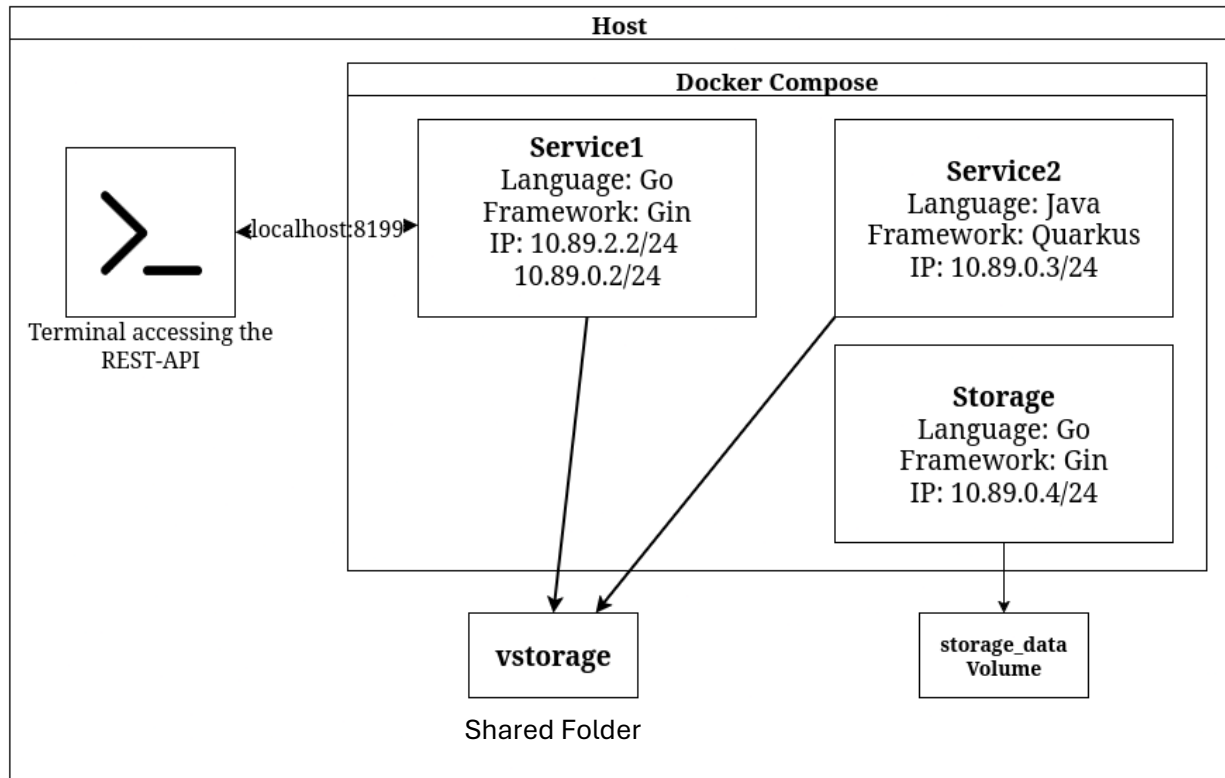
```
→ storage git:(exercise1) X docker compose --version
```

Emulate Docker CLI using podman. Create /etc/containers/nodocker to quiet msg.

```
>>>> Executing external compose provider "/usr/local/bin/docker-compose". Please see podman-  
compose(1) for how to disable this message. <<<<
```

Docker Compose version v2.32.4

Diagrams of the Architecture:



For security reasons, two networks were created to ensure that there is no access to **Storage** and **Service2** from outside. The services reach each other via the DNS names provided by Docker.

In order to make the `cat ./vstorage` command work, I shared the project root folder with **Service1** and **Service2**. The alternative would be to share the file, but then the file would have to be created beforehand; otherwise, Docker Compose would create a folder.

Analysis of the content of the status records.

Output:

```
→ devops-course git:(exercise1) X curl localhost:8199/log
→ devops-course git:(exercise1) X curl localhost:8199/status
2025-09-27T13:24:56Z: uptime 0.02140404 hours, free disk in root: 459945 MBytes
2025-09-27T13:24:56Z: uptime 0.02277778 hours, free disk in root: 459944 MBytes%
→ devops-course git:(exercise1) X curl localhost:8199/log
2025-09-27T13:24:56Z: uptime 0.02140404 hours, free disk in root: 459945 MBytes
2025-09-27T13:24:56Z: uptime 0.02277778 hours, free disk in root: 459944 MBytes
→ devops-course git:(exercise1) X cat ./vstorage
2025-09-27T13:24:56Z: uptime 0.02140404 hours, free disk in root: 459945 MBytes
2025-09-27T13:24:56Z: uptime 0.02277778 hours, free disk in root: 459944 MBytes
→ devops-course git:(exercise1) X
```

Which disc space and uptime you measured?

Both services measured the same data since they are executed on the same host. Furthermore, the disk space is almost the same only rounding issues lead to a slight difference.

How relevant are those measurements?

It's interesting to see that software deployed as a container can receive log data from the host machine. Besides that, I think the only useful thing is that the software can track if it would exceed the storage limit set by the provider and give an early warning to the customer.

What should be done better?

Logging the uptime this way is very inefficient; it would be better to use days, hours, and seconds as the format. But it depends on the use case—if it's not supposed to be read by humans and the storage taken up by the logs matters, then a different format is better.

Analysis and comparison of persistent storage

Service one and two use a common file in a shared folder. This works for this project, but when there are more write actions, it could get messy. In general, it's not recommended due to race conditions and concurrency issues. The better approach would be to share a unique folder with each service. If it has to be in a common folder, then it would at least be better to use a unique file.

For the storage, a different approach was chosen where the data is stored in a place provided by Docker. This way, the data is encapsulated from the other services since it can only be accessed through the storage service. There is also isolation from the host, but the volume can still be manipulated by the host—just not so easily. Overall, I like the approach since there is more control over access to the stored files.

Teacher's instruction for cleaning up the persistent storage

`docker-compose down --volumes`

Or if only the volume should be removed:

`docker rm -v devops-course_storage_data`

The log file is in the root of the project it can be just deleted.

What was difficult?

The instructions were not clear at first, so a lot of time was spent investigating the goal and asking questions to clarify what the expected outcome should be. A huge amount of time was spent figuring out how to share the `./vstorage` file with the containers. It turned out that a setup container that just creates the file works sometimes, but when having two containers (service1 and service2), the Docker host sometimes decides not to wait for the setup to finish and instead creates a folder in the meantime before starting service1 or service2. A lot of time was spent trying to find solutions to prevent this non-deterministic behavior, but they would most likely require external tools. This issue was eventually resolved by asking in the lecture.

With Docker, it also becomes more complex to debug the system, so working carefully is important. Especially for the Java service, it takes a long time to rebuild it for basic bug fixes.

GeorgSeng

With regard to Java, it was a bit of a challenge to get it working since the original command to build the Docker image assumes it's executed on the host. Therefore, I had to rethink my approach and build it myself in a multi-stage build, then copy the executables to the target image.

What were the main problems?

There was no specific problem since I already have a solid foundation in using Docker, even for complex issues. I would have wished the instructions were clearer. I understand that a customer also doesn't have clear instructions and it's important to ask questions, but in this setup, I would have preferred a faster interaction cycle to clarify the open questions.

Remakes on the technology used:

Service 1 / Storage

It is written in Go and uses the Gin framework (<https://gin-gonic.com/>) that provides handlers to manage responses and catch errors in case something happens during the processing of the request. I chose this technology stack because it's easy to program small services with and offers an exceptionally small resource footprint. The major downside of Go is the error handling, which can cause the code to become bloated. There are some patterns to reduce this issue, but it's not as convenient as in other languages. Additionally, the absence of advanced language features can be annoying when developing larger systems.

Service 2

It is written in Java and uses the Quarkus framework (<https://quarkus.io/>). I chose this technology stack because I'm familiar with it and because Quarkus has excellent documentation. It used to be compatible with the MicroProfile standard (<https://microprofile.io>), which is a collection of interfaces a microservice written in Java should have for interoperability with other services.

GeorgSeng

Quarkus is also an interesting framework since it pioneered the usage of GraalVM (<https://www.graalvm.org/>) for native compilation. This can drastically reduce the memory footprint and make Java or Kotlin an interesting choice for microservices. What I also love about Quarkus is that it's just a nicely coupled combination of open-source libraries, making it very easy to adapt to personal needs. Furthermore, it is awesome for debugging since you get more precise error messages than with Spring. In my opinion, the Spring framework does too much abstraction, which makes it much harder to work with.